



MATERIABILL

# Engineering Handoff

Construction project management on the Inframodern platform — full technical specification for the build team.

---

|                          |                                                                       |
|--------------------------|-----------------------------------------------------------------------|
| <b>Product</b>           | Materiabil — multi-tenant construction PM & payments                  |
| <b>Platform</b>          | Inframodern satellite app (OAuth + RabbitMQ sync)                     |
| <b>Reference pattern</b> | ShiftBill / InfraFinance (Fastify · React/Vite · Prisma · PostgreSQL) |
| <b>Status</b>            | Validated clickable prototypes + this spec → ready to build           |
| <b>Companion files</b>   | BUILD-SPEC.md · ONBOARDING-FLOWS.md · *-prototype.html                |

# Contents

---

- 1 — Executive summary
- 2 — Product overview
- 3 — Architecture & tech stack
- 4 — Data model
- 5 — Inframodern integration
- 6 — Cross-cutting business rules
- 7 — Functional spec: screens
- 8 — Key flows
- 9 — API surface
- 10 — Build plan & milestones
- 11 — Open decisions
- 12 — References & appendix
- 13 — Marketplace & reputation (roadmap)

# 1 — Executive summary

---

Materiabil is a construction project-management and payments product built as a satellite application on the **Inframodern** platform. It serves contractors, their clients (project owners), and their subcontractors in one connected network.

The defensible idea is the **network model**: a client is not a guest inside one contractor's app — they are a first-class account that spans every contractor they build with. The same recursion runs on the supply side: a subcontractor is simply a contractor whose client is the main contractor. No competitor (Procore, Buildertrend, Houzz Pro) offers this; they are all single-contractor silos.

**Build from a proven template.** InfraFinance (in the repo) is a working Inframodern satellite generated from the ShiftBill pattern. Clone its structure — OAuth, RabbitMQ consumer, Prisma mirroring — and add the construction domain.

Differentiators to preserve while building: (1) the client/sub network with one identity across firms; (2) premium editorial UI (the “Settled” design system); (3) deep commercial mechanics — cost-plus disclosure, retention, pay-when-paid, multi-currency (SAR/EGP), bilingual EN/AR; (4) the payment-continuity “goodwill” record that runs down the whole chain.

## 2 — Product overview

---

### The three actors

- **Contractor** (e.g. Essential Vision) — runs a workspace: projects, money, materials/BOM, subcontractors, approvals, documents, people & roles, branding. The primary admin app.
- **Client / owner** (e.g. Omar Khalil) — one account spanning all their contractors; approves, signs, and pays. Sees a brand-skinned portal per project.
- **Subcontractor** (e.g. Delta MEP) — a contractor in their own right whose client is the main contractor; runs their package as a project and can have their own sub-subs (recursion).

### The network, stated precisely

- Each organisation runs a workspace and is the contractor within it.
- Relationships are directional: a **client-link points up** (who hired and pays you); a **sub-link points down** (whom you hired). One org's sub-link IS the counterparty's client-link — the shared edge.
- Strict isolation: a contractor sees only their own relationship with a client/sub; only the client sees their full cross-contractor picture.
- Growth loops both ways: clients invite contractors (demand side); contractors invite subs who can become full contractors (supply side).

### Brand the project, not the app

White-label applies at the **project** level: each contractor's accent/logo/domain skins the client's view of that project, while the client's cross-contractor home stays neutral. This reconciles per-contractor branding with one unified client account.

## 3 — Architecture & tech stack

### Integration model

Materiabil plugs into Inframodern on two surfaces:

- **OAuth login** — Inframodern is the identity provider; Materiabil is an OAuth client (authorization\_code + refresh\_token). Tokens stay server-side in an encrypted session; the browser only holds a session cookie. Access is gated by app-installed-in-workspace + active subscription + user is workspace admin or allowed user.
- **RabbitMQ sync** — Materiabil consumes shared entities from Inframodern over a topic broker, mirroring Inframodern IDs (never generating local IDs for synced rows), and may publish its own entities later. Pure consumer at launch.

### Tech stack (match the reference apps)

| Layer         | Choice                              | Notes                                                       |
|---------------|-------------------------------------|-------------------------------------------------------------|
| Backend       | Fastify (Node)                      | Clone ShiftBill/InfraFinance backend structure              |
| ORM / DB      | Prisma + PostgreSQL                 | Mirrored Inframodern entities + local domain models         |
| Auth          | @fastify/passport + passport-oauth2 | Strategy named 'inframodern'; secure-session                |
| Messaging     | RabbitMQ                            | Consumer + handlers; outbox for failed publishes            |
| Frontend      | React + Vite                        | "Login with Inframodern", route guards, React Query         |
| Design system | "Settled"                           | Fraunces / Instrument Sans / Fragment Mono; copper on paper |

### Naming conventions (must match every integrated app)

|                      |                                                       |
|----------------------|-------------------------------------------------------|
| RABBITMQ_ENVIRONMENT | testing   staging   production (identical everywhere) |
| APPLICATION_NAME     | {appCode}-{env} e.g. materiabil-testing               |
| exchange             | x.{source}                                            |
| queue                | q.{source}.{consumer}.{entity}                        |
| routingKey           | {source}.{entity}                                     |
| sync completion      | {APPLICATION_NAME}.sync-completion                    |

**Three discipline rules (prevent most bugs).** Shared IDs are the contract (never re-key synced rows) · dependency-ordered sync (declare dependsOn) · always publish even an empty items:[] or the sync job hangs.

## 4 — Data model

### Synced (mirrored) entities — owned by Inframodern

Keyed by the Inframodern id; preserve deleted/deletedAt for soft-delete propagation. Consume the same base set InfraFinance does:

| Entity                | Construction reading                                                  |
|-----------------------|-----------------------------------------------------------------------|
| <b>users</b>          | Internal staff (PMs, procurement, site engineers)                     |
| <b>brands</b>         | The contractor org identity (e.g. Essential Vision) — drives branding |
| <b>locations</b>      | Optional site / warehouse anchors                                     |
| <b>customers</b>      | Project clients / owners (and signatories)                            |
| <b>taxes</b>          | VAT on POs, invoices, certificates                                    |
| <b>exchange-rates</b> | Multi-currency portfolio (SAR primary, EGP for Cairo)                 |

### Materiabillocal models (owned here; local cuid ids)

| Model                                           | Key fields                                                                                                                                                                                                                                  |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project</b>                                  | name, city, commercialModel, currency, contractValue, status, progressPct, deliveryDate, baselineDeliveryDate (immutable), pmUserId, brandId, geo, disclosureDepth, retentionPct, graceWindowMins, clientOrgId? / customerId?, originatedBy |
| <b>AgreementTerms</b>                           | model, feeType, feePct/fixedFee, targetCost, gmp+gmpAmount+savingsSplit, reimbursableCategories[], feeOnSubs, disclosureDepth, retentionPct, billingCycle — the commercial engine                                                           |
| <b>ProjectPhase / Milestone</b>                 | phases & milestones; milestone completion unlocks linked draws                                                                                                                                                                              |
| <b>DrawItem</b>                                 | money IN from client: amount, currency, status, expectedDate, releasedDate, retentionAmount                                                                                                                                                 |
| <b>Payable</b>                                  | money OUT (org-only): payeeType, amount, terms, dueDate, status, payWhenPaidDrawItemId?, retentionAmount                                                                                                                                    |
| <b>PaymentEdge</b>                              | the shared payable↔draw between two workspaces (recursion)                                                                                                                                                                                  |
| <b>BudgetLine</b>                               | category, budgetAmount, actualAmount, audience                                                                                                                                                                                              |
| <b>Material / PurchaseOrder / MaterialUsage</b> | BOM: code, status, unitPrice, qtyOrdered/Used; PO provenance; usage log                                                                                                                                                                     |
| <b>Submittal / DesignPackage / Variation</b>    | SUB/DSN/V items with revisions; approved variations immutable                                                                                                                                                                               |
| <b>Snag</b>                                     | area, status, fixPhotoid (mandatory to fix), audience, raisedBy; only client closes                                                                                                                                                         |
| <b>Document / Certificate / Signatory</b>       | category+audience docs; auto-filled certs; multi-party signing, no fixed order                                                                                                                                                              |

| Model                                                 | Key fields                                                                                           |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>SignOff</b>                                        | unifying approvals record; awaitingParty (client team); honors grace window; immutable when resolved |
| <b>FeedPost / Comment</b>                             | updates with audience (internal participants client); polymorphic comments                           |
| <b>Role / RoleAssignment</b>                          | composed from a platform permission catalog; manage_roles invariant                                  |
| <b>ContinuityRecord / PaymentEvent</b>                | built vs collected, daysCarried, status (on_track carrying paused) — goodwill                        |
| <b>ClientIdentity / ClientLink / ContractorInvite</b> | network identity: one login ↔ many workspace customer records; powers invites                        |
| <b>OutboxMessage</b>                                  | buffer for failed RabbitMQ publishes                                                                 |

**Recursion in the schema.** A project's client is either an end-customer (customerId) OR another workspace (clientOrgId). Generalize ClientIdentity/ClientLink with a role (client | subcontractor | consultant). A PaymentEdge joins a payer workspace's payable to a payee workspace's draw — pay-when-paid & goodwill propagate along edges.

## 5 — Inframodern integration

Edits inside the **inframodernbackend** repo (mirror InfraFinance's **inframodern-changes** set):

- **utils/seeder/static/applications.js** — register a *materiabil* app (code, type:management, localized name/desc, requireActivation:false, freeTrialDays:14) with an OAuth client (clientId, generated secret, grants, four redirectUris per environment). Copy the secret into the backend .env.
- **resources.json** — add the materiabil sync graph (pure consumer): users, brands, locations, taxes, exchange-rates, customers with dependsOn ordering.
- **utils/rabbitmq/config/routing.js** — add materiabil to STATUS\_SOURCES for admin sync status.
- Add Plan/PlanPrice/Feature rows if billable; confirm redirect URIs match per environment.

### resources.json snippet

```
"materiabil": {
  "publishes": [],
  "consumes": { "inframodern": {
    "users": {}, "brands": {"dependsOn":["users"]},
    "locations": {"dependsOn":["brands"]}, "taxes": {"dependsOn":["brands"]},
    "exchange-rates": {}, "customers": {"dependsOn":["brands"]} } }
}
```

**Food vs construction — no core change needed (mostly).** Inframodern's shared model is domain-neutral except the catalog/product cluster, which Materiabil simply does not consume. The one place the core may need to evolve is the cross-workspace ClientIdentity (§11, Open decisions).



## 6 — Cross-cutting business rules

---

Enforce these server-side, not just in the UI:

- **Audience / visibility** — org | participants | client on feed, docs, money notes, snags. Money-out is never client-visible.
- **Immutability** — agreed timeline baseline, approved variations, recorded approvals, executed certificates, released payments are append-only.
- **Grace window** — decisions have an undo period (org default 10 min) before they commit and notifications fan out.
- **Mandatory evidence** — a snag cannot be marked fixed without a fix photo on record.
- **Pay-when-paid** — a sub payable linked to a client draw stays flagged until that draw is released.
- **Commercial-model lock** — a project's model/terms lock after the first client-visible financial record.
- **Cost-plus disclosure depth** — category rollups vs line-level; org default, per-project override; governs BOM client visibility.
- **Roles invariant** — at least one member must always hold `manage_roles`.
- **Reason-required actions** — moving a forecast date requires a client-visible reason.
- **Multi-currency** — every amount carries a currency; portfolio rollups use synced exchange-rates.
- **Payment continuity (goodwill / justified pause)** — factually record built-vs-collected and whether work continued through late payment (goodwill) or paused (contractual right), visible identically to both sides; propagates down the chain.

## 7 — Functional spec: screens

Reference the clickable prototypes for exact layout: **materiabil-admin-app.html** (contractor), **client-network-portal.html** (client), **onboarding-prototype.html** (all entry flows).

### Contractor admin — portfolio level

| Screen                       | What it does                                                                                |
|------------------------------|---------------------------------------------------------------------------------------------|
| <b>Auth</b>                  | Login (any creds = sandbox) + 2-step signup; signup → empty workspace, login → sample       |
| <b>Overview</b>              | “Needs you / Needs client” across all projects; portfolio table; empty-state for fresh orgs |
| <b>Projects</b>              | All projects; filter by city/PM/status and by role (as main contractor vs as subcontract)   |
| <b>Payments</b>              | Portfolio cash position, monthly cash-flow, per-project; money-out org-only                 |
| <b>Approvals</b>             | Cross-project Needs you / Needs client (same SignOff records the client sees)               |
| <b>Materials / BOM</b>       | Cross-project register; client-disclosure banner driven by terms                            |
| <b>Documents</b>             | Certificates (handover/completion/payment) + project documents                              |
| <b>People &amp; roles</b>    | Members + role builder from the permission catalog; manage_roles invariant                  |
| <b>Branding &amp; domain</b> | Accent/logo/custom domain (CNAME+SSL); live client-portal preview                           |
| <b>Settings</b>              | Grace window, default retention, disclosure depth, notifications, suggestion throttle       |

### Contractor admin — project workspace tabs

| Tab                        | What it does                                                                                           |
|----------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Dashboard</b>           | Vitals, Needs you/client, settlement, cash flow, schedule, latest; subcontract banner if clientOrg set |
| <b>Feed</b>                | Site updates with audience; publish promotes to client                                                 |
| <b>Schedule / Timeline</b> | Phases, milestones (complete → unlock draw), client-agreed immutable baseline                          |
| <b>Money</b>               | Money in (draws: approve/release) + money out (payables, pay-when-paid) + continuity/goodwill + notes  |
| <b>Materials</b>           | Register, receive, log usage, issue PO; disclosure banner from terms                                   |
| <b>Subcontractors</b>      | Every trade: package, progress, paid/owed, on-platform; click → full sub package file                  |
| <b>Submittals / Design</b> | SUB/DSN, revisions, priced variations                                                                  |
| <b>Snags</b>               | Punch list; mandatory fix photo; only client closes                                                    |
| <b>Docs</b>                | Project documents incl. auto-generated contract; site photos                                           |
| <b>Setup</b>               | Agreement terms (commercial engine), structure, participants; invite org/client                        |

### Client portal (mobile-first)

- 
- Neutral “My projects” home spanning all contractors; each project view skins to that contractor's brand.
  - Per project: Needs you (approve/sign/decide), your account (paid/due), build continuity (goodwill + pay), latest.
  - Self-serve start: empty home → add a project stub → invite the contractor (project-first).
  - Isolation banner: only the client sees all projects together.

## 8 — Key flows

---

### Create project (model-aware)

4 steps: Basics → Model → Terms → Review. The Terms step adapts to the model. Cost-plus captures fee basis (% or fixed), target cost, GMP + savings split, reimbursable categories, fee-on-sub, disclosure depth, retention, billing cycle. On create, terms are stored on the project and rendered into a contract Document; they govern BOM disclosure, budget, billing, and lock after the first client-visible record.

### Onboarding (see ONBOARDING-FLOWS.md)

- Contractor — self-serve (marketplace install) or pulled in by a client invite.
- Client — invited: invite landing → verify identity (creates ClientIdentity) → lands in project.
- Client — self-serve (project-first): sign up → verify → add a project stub → invite the contractor → they claim it.
- Client invites a contractor — two variants (stub vs contractor-builds); links by verified email/phone.

### Recursion & the shared edge

A subcontractor runs their package as a project in their own workspace, with the main contractor as the client. The contractor's payable to the sub and the sub's incoming draw are one PaymentEdge — pay-when-paid and the goodwill/continuity record run along it down the chain. Demonstrated in the prototype: workspace switch → Delta MEP Co.

## 9 — API surface (indicative)

REST, session-authenticated, workspace-scoped on every query. Client/sub reads filtered by ClientLink (isolation). Indicative endpoints — expand per resource:

```
GET /auth/callback/inframodern    OAuth callback (sets session)
POST /auth/logout
GET /user                          rehydrate session

GET /projects      ?role=main|sub  list (workspace-scoped)
POST /projects      create (+ AgreementTerms, contract doc)
GET /projects/:id   detail
GET /projects/:id/draws POST .../draws/:id/approve /release
GET /projects/:id/payables POST .../payables/:id/pay
GET /projects/:id/materials POST .../materials /receive /use
GET /projects/:id/subcontractors
GET /projects/:id/snags POST .../snags/:id/assign /fix (photo required)
GET /projects/:id/signoffs POST .../signoffs/:id/respond
POST /invites      (client | org | sub) – verified-contact linking
GET /continuity/:projectId      built-vs-collected + goodwill
```

**Server-side enforcement.** All §6 rules live in route handlers, backed by an append-only audit/event log (“logged in the trail”) and a grace-window queue that delays commit + notifications.

## 10 — Build plan & milestones

| Phase                       | Scope                                                                                    | Outcome                                                  |
|-----------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------|
| 0 • Decisions               | Resolve open decisions (§11): participant-org boundary, client-identity A/B              | Schema locked                                            |
| 1 • Scaffold                | Clone InfraFinance: Fastify+Prisma+RabbitMQ+OAuth; React/Vite shell; inframodern-changes | Login with Inframodern works; can install in a workspace |
| 2 • Sync                    | sources.js + importers for the 6 consumed entities; mirror IDs; outbox                   | Users/brands/locations/customers/taxes/FX synced         |
| 3 • Core domain             | Project + AgreementTerms + phases/milestones + documents; create-project flow            | Create & view a project end-to-end                       |
| 4 • Money                   | Draws, payables, retention, pay-when-paid, budget, continuity/goodwill                   | Money in/out with the rules enforced                     |
| 5 • Site & quality          | Feed, materials/BOM, submittals/design, snags (fix-photo)                                | Day-to-day site ops                                      |
| 6 • Client portal           | Client identity, network home, brand-per-project, approvals/sign-offs                    | Clients approve/sign/pay; isolation enforced             |
| 7 • Network & recursion     | Sub invites, clientOrg projects, PaymentEdge, role filter                                | Subcontract chain works both directions                  |
| 8 • Roles/branding/settings | Permission catalog + invariant, white-label + custom domain                              | Org self-service                                         |
| 9 • Mobile & polish         | Native-ish field capture (camera/offline), i18n/RTL, notifications                       | Field-ready                                              |

**Sequencing note.** Build screens outward from the data model; reuse the prototype HTML/CSS as the design reference. Overview and Payments aggregate everything, so build them after the project domain exists.

## 11 — Open decisions (resolve before/at scaffold)

---

- **Client identity — Option A vs B.** A: a shared ClientIdentity in Inframodern core (cleanest long-term, a genuine core change). B: Materiabill owns a directory mapping one verified login to many workspace customer records (faster to ship). Recommendation: B for v1, keep the schema A-ready.
- **Participant-org boundary.** Are subcontractor/consultant orgs Inframodern guest brands/workspaces, or Materiabill-local ParticipantOrg rows (with nullable inframodernBrandId)? Affects the whole schema.
- **Stub ownership.** A client-created project originates client-side and is a thin stub until a contractor claims it into their workspace — Project needs originatedBy + nullable owning workspace until claimed.
- **Billing.** Plans/prices/features if Materiabill is itself billable on Inframodern.
- **Disclosure computation.** How line-level cost+fee actually computes from BOM into client-visible draws.

## 12 — References & appendix

### Companion files (in the project folder)

| File                              | Contents                                                                                    |
|-----------------------------------|---------------------------------------------------------------------------------------------|
| <b>BUILD-SPEC.md</b>              | Full data model, integration, recursion (§6.2), agreement terms, continuity, per-screen map |
| <b>ONBOARDING-FLOWS.md</b>        | Contractor/client/self-serve onboarding + invite-loop variants + identity linking           |
| <b>UI-UX-MARKET-COMPARISON.md</b> | Benchmark vs Procore/Buildertrend/Houzz/CompanyCam/Fieldwire + strategy                     |
| <b>materiabil-admin-app.html</b>  | Working contractor admin prototype (data-driven, 2 workspaces)                              |
| <b>client-network-portal.html</b> | Client network portal prototype (multi-contractor, self-serve)                              |
| <b>onboarding-prototype.html</b>  | Persona-switcher walkthrough of every onboarding flow                                       |

### Inframodern source references

- [inframodernbackend](#) (prisma/schema.prisma, routes/oauth, models/OAuth.js, resources.json, rabbitmq/\*)
- [shiftbillbackend / shiftbillfrontend](#) — canonical reference implementation
- [infrafinance/](#) (in repo) — the closest working template to clone
- “Inframodern New App Integration Guide” §1–§15 — the runbook this spec follows



## 13 — Marketplace & reputation (roadmap)

Build-later, design-for-now. The product is already a verified relationship graph with real transactions — the hard part of any marketplace. A few decisions now keep a ratings marketplace a natural extension rather than a rewrite.

### Already in place (the foundation)

- **The network graph** — every org is a node; clients and subs are first-class identities linked across many counterparties.
- **Verified identities + invites** — one identity per verified email/phone makes ratings trustworthy: only real counterparties of a real project can review.
- **Reputation inputs already captured** — ContinuityRecord (goodwill / pays-on-time) and sub compliance (insurance, license, prequalification) are two-sided signals.
- **Multi-directional by recursion** — client→contractor, contractor→sub, sub→contractor, plus a client reputation (payment reliability).

### Design-for-now (cheap now, expensive to retrofit)

- **Keep reputation & identity at the global org/identity level**, not workspace-scoped, so track record travels — the marketplace-ready reason to favour client-identity Option A (\$11).
- Add two thin models now: OrgProfile and an engagement-scoped Review (below).
- **Consent & isolation are the guardrail** — only opt-in / aggregate data becomes public; private relationships never leak.
- Treat compliance + continuity as live reputation feeds into the profile from day one.

### Model sketch — OrgProfile

| Field               | Notes                                                                                        |
|---------------------|----------------------------------------------------------------------------------------------|
| orgId               | FK to the org/brand (workspace owner)                                                        |
| displayName, about  | Public-facing identity, distinct from the workspace                                          |
| trades[], regions[] | For discovery / filtering later                                                              |
| portfolio[]         | Case studies (project refs + photos) with per-item visibility/consent                        |
| trackRecord         | Derived: projectsCompleted, totalValue, onTimePct, neverPausedStreak, avgRating, ratingCount |
| verification        | licenseVerified, insuranceValid, prequalification, KYC level (fed by sub-compliance)         |
| visibility          | public   unlisted   private                                                                  |

### Model sketch — Review (engagement-scoped)

| Field        | Notes                                                                    |
|--------------|--------------------------------------------------------------------------|
| engagementId | The specific project + relationship reviewed (no engagement → no review) |

| Field                               | Notes                                                                                            |
|-------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>direction</b>                    | client_to_contractor   contractor_to_sub   sub_to_contractor                                     |
| <b>reviewer / subject / project</b> | Who reviews whom, on which job                                                                   |
| <b>scores</b>                       | overall + dimensions: quality, schedule, communication, payment behaviour                        |
| <b>comment, response</b>            | Reviewer text; the subject may post one reply                                                    |
| <b>constraints</b>                  | One per engagement; unlocked after a verifiable milestone; edit window; status published pending |

**Deferred (sits on top of the graph later).** Public discovery/directory & search, RFQ / tender / bidding, lead matching, escrow/marketplace payments — none require schema rework if the above is respected.

Generated for the MateriabiLL build team. Pair this PDF with the markdown specs and the clickable prototypes; the prototypes are the source of truth for layout and interaction, this document and BUILD-SPEC.md for the data model and rules.